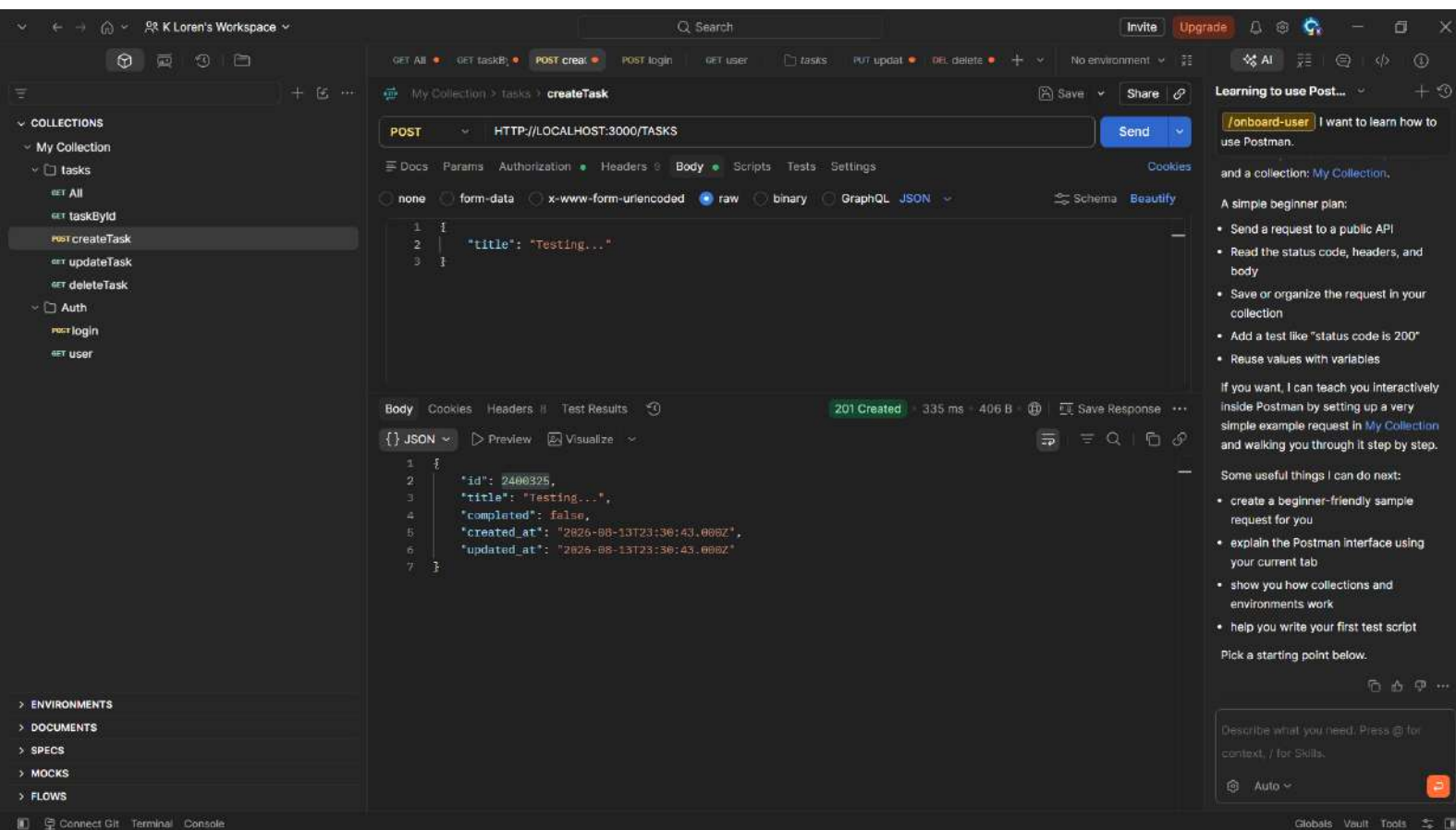




K Loren's Workspace

My Collection > tasks > deleteTask

SaveShare

DELETEhttp://localhost:3000/tasks/400325

Send

DocsParamsAuthorizationHeadersBodyScriptsTestsSettings

Auth TypeBearer TokenToken

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

BodyCookiesHeadersTest Results

200 OK1.92 s306 B

Save Response

JSONPreviewVisualize

```
1 {
2   "message": "Task deleted successfully"
3 }
```

COLLECTIONS

My Collection

tasks

GET All

GET taskById

POST createTask

GET updateTask

DELETE deleteTask

Auth

POST login

GET user

ENVIRONMENTS

DOCUMENTS

SPECS

MOCKS

FLOWS

Connect GitTerminalConsole

Learning to use Postman

/onboard-user

I want to learn how to use Postman.

and a collection: [My Collection](#).

A simple beginner plan:

- Send a request to a public API
- Read the status code, headers, and body
- Save or organize the request in your collection
- Add a test like "status code is 200"
- Reuse values with variables

If you want, I can teach you interactively inside Postman by setting up a very simple example request in [My Collection](#) and walking you through it step by step.

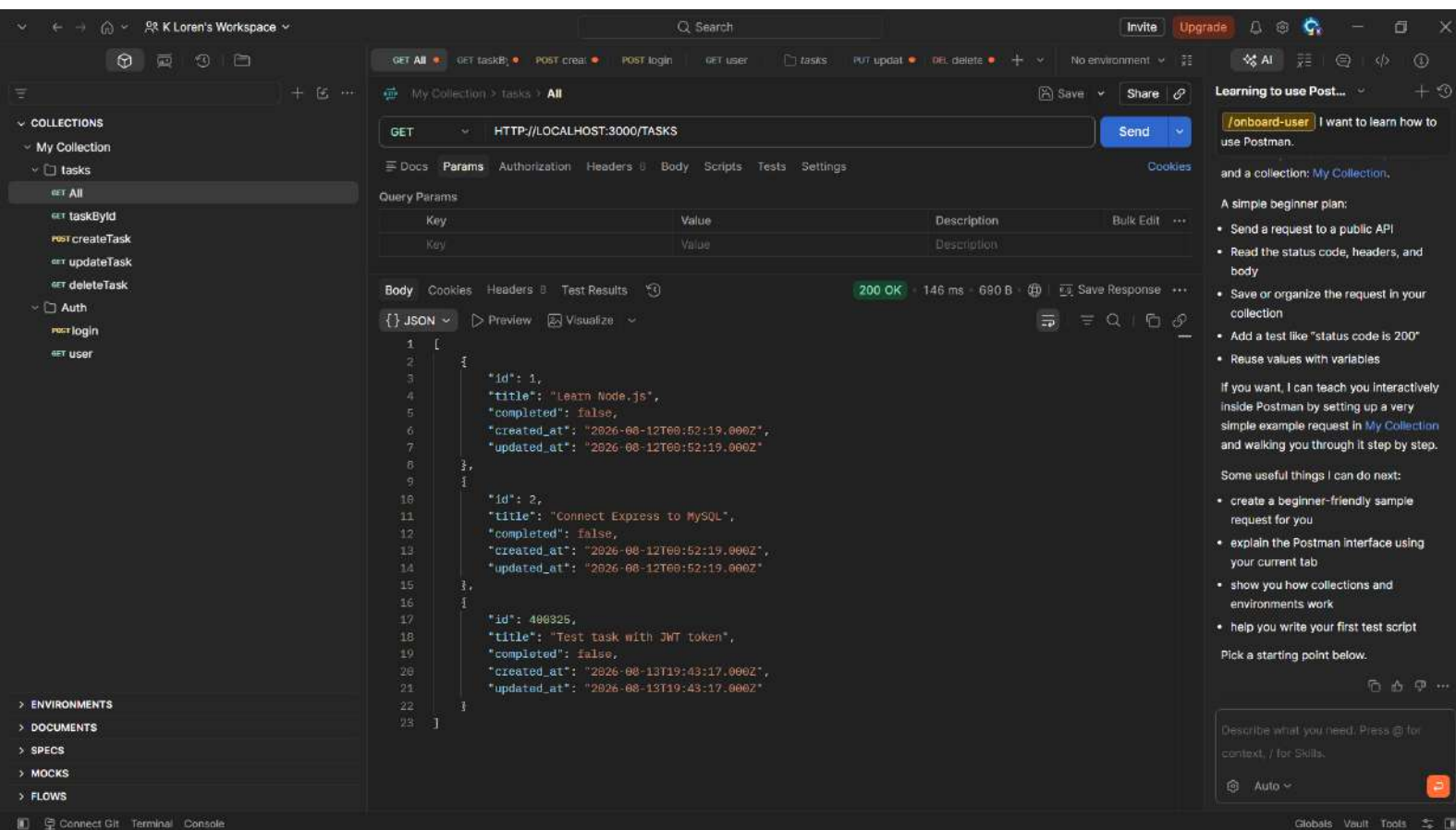
Some useful things I can do next:

- create a beginner-friendly sample request for you
- explain the Postman interface using your current tab
- show you how collections and environments work
- help you write your first test script

Pick a starting point below.

Describe what you need. Press @ for context, / for Skills.

Auto



⌵

←

→

🏠

K Loren's Workspace

🔍 Search

Invite

Upgrade

🔔

⚙️

🌐

—

📦

✕

📁

+

🔗

⋮

COLLECTIONS

My Collection

tasks

GET All

GET taskById

POST createTask

PUT updateTask

DELETE deleteTask

Auth

POST login

GET user

> ENVIRONMENTS

> DOCUMENTS

> SPECS

> MOCKS

> FLOWS

CONNECT GIT

TERMINAL

CONSOLE

GET All

GET taskB...

POST crea...

POST login

GET user

tasks

PUT updat...

DEL delete

+

No environment

⋮

AI

🔧

💬

📄

🔔

My Collection > tasks > All

Save

Share

GET

HTTP://LOCALHOST:3000/TASKS

Send

Docs

Params

Authorization

Headers

Body

Scripts

Tests

Settings

Cookies

Query Params

Key

Value

Description

Bulk Edit

...

Body

Cookies

Headers

Test Results

🔔

200 OK

93 ms

677 B

🌐

📄

Save Response

⋮

{} JSON

▶ Preview

🖼 Visualize

⌵

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

[

{

"id": 1,

"title": "Learn Node.js",

"completed": false,

"created_at": "2026-08-12T00:52:19.000Z",

"updated_at": "2026-08-12T00:52:19.000Z"

}

,

{

"id": 2,

"title": "Connect Express to MySQL",

"completed": false,

"created_at": "2026-08-12T00:52:19.000Z",

"updated_at": "2026-08-12T00:52:19.000Z"

}

,

{

"id": 2400325,

"title": "Testing...",

"completed": false,

"created_at": "2026-08-13T23:38:43.000Z",

"updated_at": "2026-08-13T23:38:43.000Z"

}

]

Learning to use Post...

+

🔔

onboard-user

I want to learn how to use Postman.

and a collection: My Collection.

A simple beginner plan:

• Send a request to a public API

• Read the status code, headers, and body

• Save or organize the request in your collection

• Add a test like "status code is 200"

• Reuse values with variables

If you want, I can teach you interactively inside Postman by setting up a very simple example request in My Collection and walking you through it step by step.

Some useful things I can do next:

• create a beginner-friendly sample request for you

• explain the Postman interface using your current tab

• show you how collections and environments work

• help you write your first test script

Pick a starting point below.

Describe what you need. Press @ for context, / for Skills.

🔍 Auto

🔔

Globals

Vault

Tools

⚙️

📄

C:\Users\User\Documents\OJT Activities\SQL_Activity3\server.js (SQL_Activity3) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

FOLDERS

- SQL_Activity3
 - controllers
 - /* authController.js
 - /* taskController.js
 - middleware
 - /* authenticateToken.js
 - models
 - /* Tasks.js
 - /* User.js
 - node_modules
 - routes
 - /* authRoutes.js
 - d
 - H
 - .env
 - .gitignore
 - /* database.js
 - /* package-lock.json
 - /* package.json
 - <> README.md
 - /* schema.sql
 - Screenshot_Comp.pdf
 - /* server.js
 - /* utils.js

server.js

```
11
12 app.use(express.json());
13 app.use(cors());
14
15 // Mount Authentication Routes (/auth/register and /auth/login)
16 app.use('/auth', authRoutes);
17
18 app.get("/", (request, response) => {
19   response.json({ message: "Task API is running" });
20 });
21
22 // PUBLIC: Get all tasks from MySQL
23 app.get("/tasks", taskController.getAllTasks);
24
25 app.get("/tasks/:id", taskController.getTaskById);
26
27 // app.post(route, middleware, controller)
28
29 app.post("/tasks", authenticateToken, taskController.createTasks);
30
31 // PROTECTED: Update a task in MySQL (Requires valid JWT)
32 app.put("/tasks/:id", authenticateToken, taskController.updateTask);
33
34 // PROTECTED: Delete a task from MySQL (Requires valid JWT)
35 app.delete("/tasks/:id", authenticateToken, taskController.deleteTask);
36
37 async function startServer() {
38   try {
39     const connection = await pool.getConnection();
40     console.log("Connected to MySQL successfully");
41     connection.release();
42
43     app.listen(PORT, () => {
44       console.log(`Server is running at http://localhost:${PORT}`);
45     });
46   } catch (error) {
47     console.error("Unable to connect to MySQL:", error.message);
48     process.exit(1);
49   }
50 }
```

Line 5, Column 34

main (13) Spaces: 2 JavaScript

C:\Users\User\Documents\OJT Activities\SQL_Activity3\controllers\taskController.js (SQL_Activity3) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

FOLDERS

- SQL_Activity3
 - controllers
 - authController.js
 - taskController.js
 - middleware
 - authenticateToken.js
 - models
 - Tasks.js
 - User.js
 - node_modules
 - routes
 - authRoutes.js
 - d
 - H
 - .env
 - .gitignore
 - database.js
 - package-lock.json
 - package.json
 - README.md
 - schema.sql
 - Screenshot_Comp.pdf
 - server.js
 - utils.js

```
33 const createTasks = async (request, response) => {
34   try {
35     const { title } = request.body;
36     if (typeof title !== "string" || !title.trim()) {
37       return response.status(400).json({ message: "title is required" });
38     }
39     const [result] = await pool.execute(
40       "INSERT INTO tasks (title) VALUES (?)",
41       [title.trim()]
42     );
43     const [tasks] = await pool.execute(
44       "SELECT id, title, completed, created_at, updated_at FROM tasks WHERE id = ?",
45       [result.insertId]
46     );
47     response.status(201).json(utils.formatTask(tasks[0]));
48   } catch (error) {
49     console.error(error);
50     response.status(500).json({ message: "Unable to create task" });
51   }
52 }
53
54
55 const updateTask = async (request, response) => {
56   try {
57     const id = Number(request.params.id);
58     const { title, completed } = request.body;
59     const [existingTasks] = await pool.execute(
60       "SELECT id, title, completed FROM tasks WHERE id = ?",
61       [id]
62     );
63     if (existingTasks.length === 0) {
64       return response.status(404).json({ message: "Task not found" });
65     }
66     const currentTask = existingTasks[0];
67     let updatedTitle = currentTask.title;
68     let updatedCompleted = Boolean(currentTask.completed);
69
70     if (title !== undefined) {
71       if (typeof title !== "string" || !title.trim()) {
72         return response.status(400).json({ message: "title must be a non-empty string" });
73       }
74       updatedTitle = title.trim();
75     }
76
77     if (completed !== undefined) {
78       if (typeof completed !== "boolean") {
79         return response.status(400).json({ message: "Completed must be a boolean" });
80       }
81       updatedCompleted = completed;
82     }
83
84     await pool.execute(
85       "UPDATE tasks SET title = ?, completed = ? WHERE id = ?",
86       [updatedTitle, updatedCompleted, id]
87     );
88   }
89 }
```

Line 118, Column 31

main (1) Spaces: 2 JavaScript

C:\Users\User\Documents\OJT Activities\SQL_Activity3\controllers\taskController.js (SQL_Activity3) - Sublime Text (UNREGISTERED)

File Edit Selection Find View Goto Tools Project Preferences Help

FOLDERS

- SQL_Activity3
 - controllers
 - authController.js
 - taskController.js
 - middleware
 - authenticateToken.js
 - models
 - Tasks.js
 - User.js
 - node_modules
 - routes
 - authRoutes.js
 - d
 - H
 - .env
 - .gitignore
 - database.js
 - package-lock.json
 - package.json
 - README.md
 - schema.sql
 - Screenshot_Comp.pdf
 - server.js
 - utils.js

```
75     return response.status(400).json({ message: "Title must be a non-empty string" });
76   }
77   updatedTitle = title.trim();
78 }
79
80 if (completed !== undefined) {
81   if (typeof completed !== "boolean") {
82     return response.status(400).json({ message: "Completed must be a boolean" });
83   }
84   updatedCompleted = completed;
85 }
86
87 await pool.execute(
88   "UPDATE tasks SET title = ?, completed = ? WHERE id = ?",
89   [updatedTitle, updatedCompleted, id]
90 );
91
92 const [tasks] = await pool.execute(
93   "SELECT id, title, completed, created_at, updated_at FROM tasks WHERE id = ?",
94   [id]
95 );
96 response.json(utils.formatTask(tasks[0]));
97 } catch (error) {
98   console.error(error);
99   response.status(500).json({ message: "Unable to update task" });
100 }
101 };
102
103 const deleteTask = async (request, response) => {
104   try {
105     const id = Number(request.params.id);
106     const [result] = await pool.execute(
107       "DELETE FROM tasks WHERE id = ?",
108       [id]
109     );
110
111     if (result.affectedRows === 0) {
112       return response.status(404).json({ message: "Task not found" });
113     }
114
115     response.json({ message: "Task deleted successfully" });
116   } catch (error) {
117     console.error(error);
118     response.status(500).json({ message: "Unable to delete task" });
119   }
120 };
121
122 const taskController = {
123   getAllTasks,
124   getTaskById,
125   createTasks,
126   updateTask,
127   deleteTask
128 };
129
130 export default taskController;
```

Line 118, Column 31

main (1) Spaces: 2 JavaScript